Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 12

Producer, Consumer and the Other Tools

Track A: the real OS, from the outside (Linux)

Elaborated by:
st. gr. FAF-241
Ermicev Sofia

Verified by:
Patricia Reițman

Chișinău – 2026

Contents

1 Introduction **2**

2 Part 1. Producer and Consumer **2**
2.1 Implementation 2
2.2 Actual execution 2
2.3 Experiment: wrong synchronization order 3
2.4 Observe answers 4

3 Part 2. The Condition Variable **4**
3.1 Implementation 4
3.2 Actual executions 4
3.3 Observe answers 5

4 Part 3. Readers and Writers **5**
4.1 Implementation 5
4.2 Actual execution 5
4.3 Observe answers 6

5 Part 4. Build a Deadlock, Then Remove It **6**
5.1 Original deadlock 6
5.2 Experiment without sleeps: ten actual runs 6
5.3 Fixed version 7
5.4 Observe answers 7

6 Conclusions **8**

1. Introduction

The objective of Laboratory Work No. 12 is to investigate coordination between concurrent threads using POSIX semaphores, mutexes, and condition variables. Four experiments are performed: a bounded producer–consumer buffer, message ordering through a condition variable, a readers–writers synchronization scheme, and a deliberate deadlock followed by its correction.

The programs were compiled and executed in Ubuntu under WSL2 in `/home/sofia/os-lab12` using GCC 13.3.0. Each program was compiled with `gcc -Wall -pthread`. The terminal excerpts in this report are the student's actual results as supplied during the laboratory work, not simulated screenshots. The submission includes four final C programs and three experimental variants.

2. Part 1. Producer and Consumer

2.1. Implementation

The shared circular buffer contains four slots (`SIZE = 4`) and transfers eight integers (`ITEMS = 8`). The semaphore `empty` begins at four, indicating the available slots; `full` begins at zero, indicating no items are ready. A mutex, `lock`, protects the buffer indices and the displayed count.

```
// Producer: reserve capacity before entering the critical section.
sem_wait(&empty);
pthread_mutex_lock(&lock);
buf[in] = item;
in = (in + 1) % SIZE;
count++;
pthread_mutex_unlock(&lock);
sem_post(&full);

// Consumer: wait for data before entering the critical section.
sem_wait(&full);
pthread_mutex_lock(&lock);
int item = buf[out];
out = (out + 1) % SIZE;
count--;
pthread_mutex_unlock(&lock);
sem_post(&empty);
```

2.2. Actual execution

```
$ gcc -Wall -pthread -o prodcons prodcons.c
$ ./prodcons
put 1    (in buffer: 1)
put 2    (in buffer: 2)
put 3    (in buffer: 3)
put 4    (in buffer: 4)
            take 1   (in buffer: 3)
put 5    (in buffer: 4)
            take 2   (in buffer: 3)
put 6    (in buffer: 4)
            take 3   (in buffer: 3)
put 7    (in buffer: 4)
            take 4   (in buffer: 3)
put 8    (in buffer: 4)
            take 5   (in buffer: 3)
            take 6   (in buffer: 2)
            take 7   (in buffer: 1)
            take 8   (in buffer: 0)
```

The producer transferred all eight items, the consumer removed items 1 through 8 in order, and the occupancy remained between zero and four.

2.3. Experiment: wrong synchronization order

In a separate file `prodcons_wrong.c`, the first two calls in `producer()` were reversed:

```
pthread_mutex_lock(&lock); // WRONG: holds lock while waiting
sem_wait(&empty);
```

```
$ gcc -Wall -pthread -o prodcons_wrong prodcons_wrong.c
$ timeout 5 ./prodcons_wrong
put 1    (in buffer: 1)
put 2    (in buffer: 2)
put 3    (in buffer: 3)
put 4    (in buffer: 4)
            take 1   (in buffer: 3)
put 5    (in buffer: 4)
$ echo "exit code: $?"
exit code: 124
```

The program timed out after five seconds. The producer held the mutex while blocked on `empty`; the consumer could not obtain that mutex to remove an item and signal `empty`. This is a deadlock.

2.4. Observe answers

1. `sem_wait(&empty)` prevents the producer from adding an item when the four-slot buffer is full. It blocks until the consumer calls `sem_post(&empty)`.
2. The consumer waits initially, because `full` is initialized to zero. The waiting call is `sem_wait(&full)`.
3. When the producer locks the mutex before waiting on `empty`, it may sleep while retaining the mutex. With a full buffer, the consumer requires the same mutex to free a slot, causing circular waiting. The observed command ended with status 124 due to `timeout`.

3. Part 2. The Condition Variable

3.1. Implementation

The program `condvar.c` coordinates a reader and writer with a shared state variable `ready`, a mutex, and a POSIX condition variable `changed`. The writer writes `hello`, sets `ready = 1`, and signals. The reader waits while the shared state indicates that the message is unavailable.

```
// Writer holds the mutex while updating the condition.
pthread_mutex_lock(&lock);
strcpy(message, "hello");
ready = 1;
pthread_cond_signal(&changed);
pthread_mutex_unlock(&lock);

// Reader waits atomically releasing/reacquiring the mutex.
pthread_mutex_lock(&lock);
while (!ready)
    pthread_cond_wait(&changed, &lock);
printf("B: I read \"%s\"\n", message);
pthread_mutex_unlock(&lock);
```

3.2. Actual executions

```
$ gcc -Wall -pthread -o condvar condvar.c
$ ./condvar
B: not ready, I sleep
A: message written
B: I read "hello"
```

```
$ gcc -Wall -pthread -o condvar_if condvar_if.c
$ ./condvar_if
B: not ready, I sleep
A: message written
B: I read "hello"
```

The experimental file `condvar_if.c` replaces `while (!ready)` with `if (!ready)`. It also completed successfully in this single observed run.

3.3. Observe answers

1. The ordinary integer variable `ready` stores the condition; the condition variable itself does not store the message-ready state.
2. In this test, replacing `while` with `if` still produced the correct sequence and read `hello`.
3. A condition wait can return spuriously, or another thread may change the shared state before the waiting thread reacquires the mutex. A `while` loop rechecks the predicate and avoids proceeding while `ready == 0`.

4. Part 3. Readers and Writers

4.1. Implementation

Three reader threads and two writer threads access a shared integer. The semaphore `rlock` serializes changes to the number of active readers; `room` grants exclusive access to a writer or to the current group of readers.

```
// First reader excludes writers; last reader admits writers.
sem_wait(&rlock);
if (++readers == 1)
    sem_wait(&room);
sem_post(&rlock);
/* reading */
sem_wait(&rlock);
if (--readers == 0)
    sem_post(&room);
sem_post(&rlock);
```

4.2. Actual execution

```
$ gcc -Wall -pthread -o readers readers.c
$ ./readers
```

```
writer 1 writes 10 (alone)
reader 1 reads 10     (readers inside: 1)
reader 2 reads 10     (readers inside: 2)
reader 3 reads 10     (readers inside: 3)
writer 2 writes 20 (alone)
```

All three reader threads were able to read the value 10 concurrently, while writers accessed the shared value without active readers.

4.3. Observe answers

1. The conditional actions `if (++readers == 1) sem_wait(&room)` and `if (--readers == 0) sem_post(&room)` ensure the first reader blocks writers and the last reader lets them in.
2. `rlock` protects increments, decrements, and tests of the `readers` count. Without it, concurrent updates could be lost, causing incorrect admission or release of the room.
3. If readers continue arriving before the last current reader exits, the writer can be delayed indefinitely. This is called *writer starvation*.

5. Part 4. Build a Deadlock, Then Remove It

5.1. Original deadlock

The program `deadlock.c` originally had two threads with opposite lock acquisition orders. Thread one acquired A and then tried B; thread two acquired B and then tried A. One-second sleeps made the circular waiting easier to reproduce.

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 ./deadlock
one: I have A, I want B
two: I have B, I want A
$ echo "exit code: $?"
exit code: 124
```

The deadlock forms the cycle: `thread one -> mutex B -> thread two -> mutex A -> thread one`.

5.2. Experiment without sleeps: ten actual runs

A copy named `deadlock_nosleep.c` removed both calls to `sleep(1)`. Each execution was bounded by `timeout 2`. The recorded exit codes were:

Run	1	2	3	4	5	6	7	8	9	10
Exit	124	0	0	0	0	0	124	0	0	0

In runs 1 and 7, the output showed both threads holding their first mutex and waiting for the second, followed by `exit code: 124`. Runs 2–6 and 8–10 printed both `I have both` messages and `done`, followed by `exit code: 0`. Consequently, 2 out of 10 runs timed out and 8 completed. Removing the sleeps did not make the unsafe acquisition order correct; it made the failure intermittent.

5.3. Fixed version

The final `deadlock.c` makes both threads acquire mutex A before B and release B before A. In `two()`, the corrected synchronization is:

```
pthread_mutex_lock(&A);
printf("two: I have A, I want B\n");
sleep(1);
pthread_mutex_lock(&B);
printf("two: I have both\n");
pthread_mutex_unlock(&B);
pthread_mutex_unlock(&A);
```

```
$ gcc -Wall -pthread -o deadlock deadlock.c
$ timeout 5 ./deadlock
one: I have A, I want B
one: I have both
two: I have A, I want B
two: I have both
done
$ echo "exit code: $?"
exit code: 0
```

The program finished normally, confirming the correction in the observed run.

5.4. Observe answers

1. In the original deadlock, thread one holds A and waits for B, while thread two holds B and waits for A. The wait-for cycle is `T1 -> B -> T2 -> A -> T1`.

2. Without the sleeps, the program did not always hang: runs 1 and 7 timed out; eight other runs completed. This is worse for debugging because the same unsafe program sometimes appears correct, depending on scheduling.

3. Acquiring locks in the same global order (A before B) eliminates the circular-wait condition: a thread cannot hold B while requesting A, so this two-lock cycle cannot form.

6. Conclusions

The four experiments illustrate several distinct synchronization mechanisms and failure modes. Counting semaphores bounded producer–consumer occupancy to four, while a mutex protected buffer changes. A reversed wait/lock order caused a reproducible deadlock. A condition variable coordinated access to a shared message, with its predicate stored separately in `ready` and checked in a loop. The readers–writers example allowed concurrent reading but exclusive writing, with possible writer starvation under a sustained reader workload. Finally, opposite mutex orders caused deadlock in all tests with deliberate delays and in 2 of 10 executions without delays; consistent ordering prevented deadlock in the final execution. The tested files and actual outputs demonstrate both correct coordination and the consequences of synchronization mistakes.